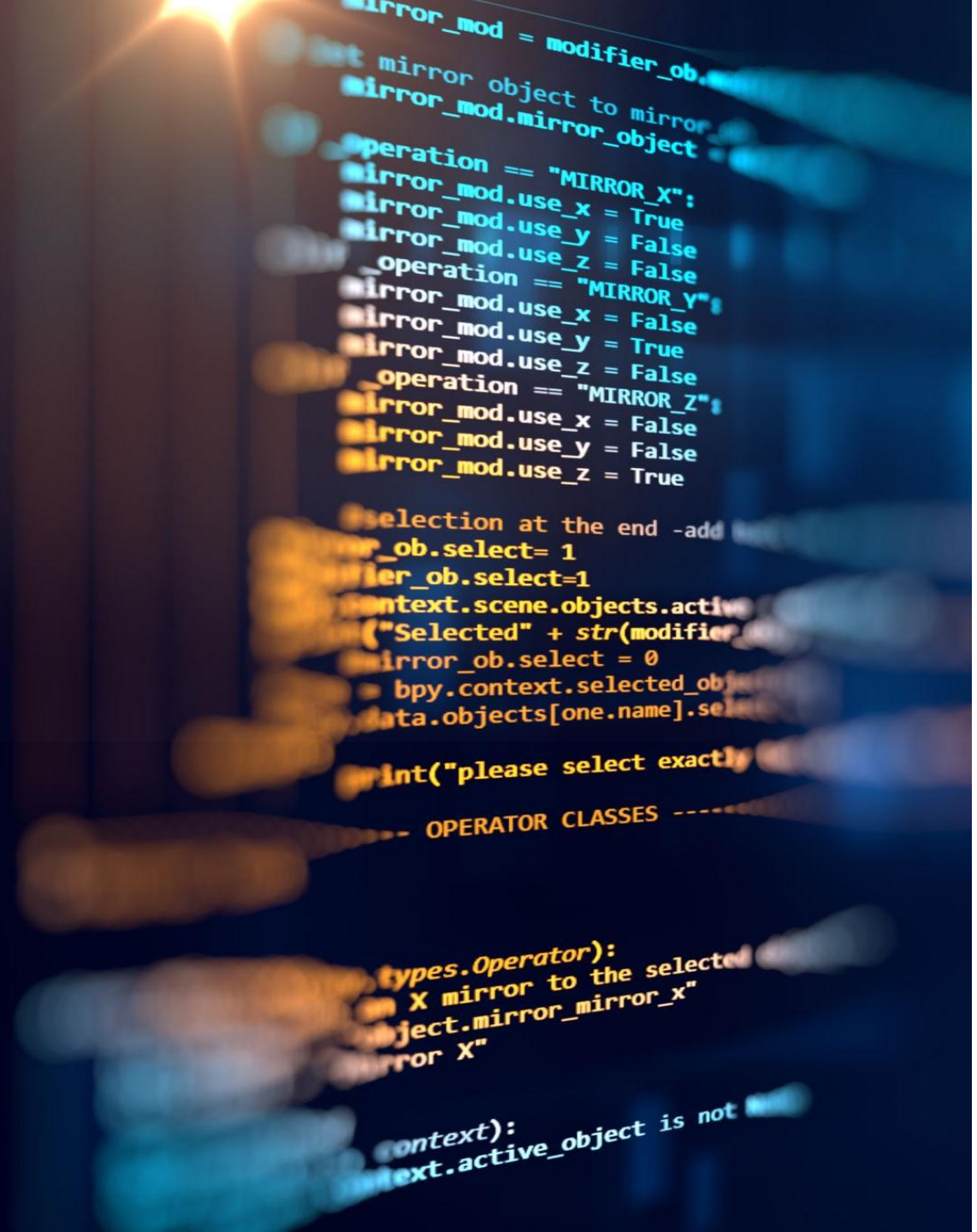
Shree Ramswaroop Memorial University



JavaScript Assignment 3

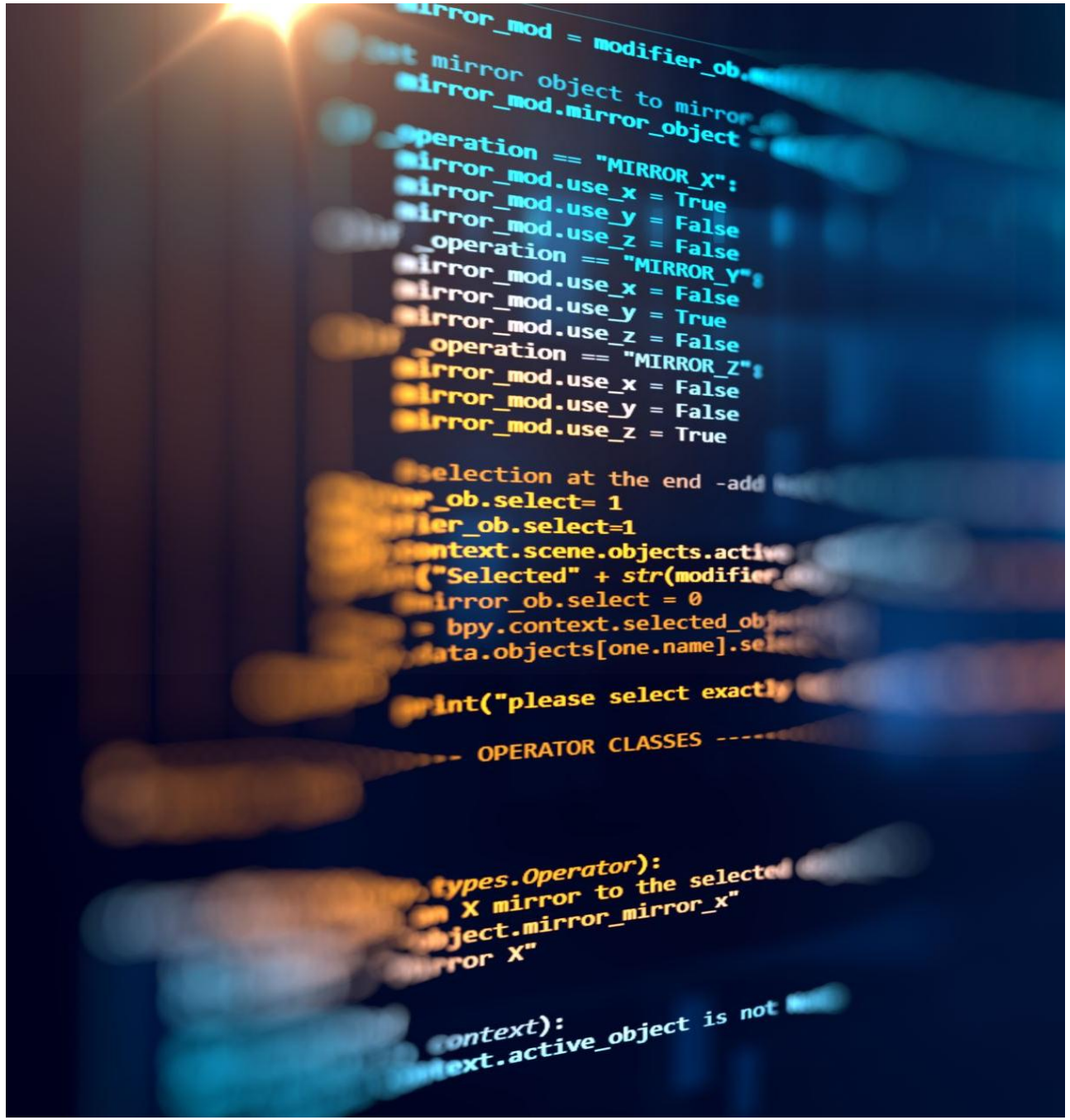
Submitted to : Kshitiz Srivastav

Submitted By : Mukesh Prajapati
Branch : B . tech (cs industry integrated)
Roll-No : 202410101230007



Q1. How can you mitigate security risks when using JavaScript on web pages?

- *To mitigate JavaScript security risks:*
- *Prevent XSS – Sanitize user input, escape output, and use Content Security Policy (CSP).*
- *Prevent CSRF – Use CSRF tokens and SameSite cookies.*
- *Secure Data – Use HTTPS, avoid storing sensitive data in localStorage.*
- *Restrict JS Execution – Avoid eval(), use strict mode, limit third-party scripts.*
- *Secure APIs – Validate inputs, use proper authentication.*
- *Update Dependencies – Keep libraries updated and audit for vulnerabilities.*



- ***Q2. Describe the importance of placing the <script> tag just before the closing </body> tag in HTML documents. How does this benefit web page performance?***

Placing the <script> tag just before the closing </body> tag improves web page performance by:

1.Faster Page Rendering – The browser loads and displays HTML content before executing JavaScript, reducing delays in rendering.

2.Non-blocking Execution – Since scripts are loaded after the main content, they don't block the parsing of HTML and CSS.

3.Better User Experience – Users see the page content faster instead of waiting for scripts to load.

4.Efficient Resource Loading – JavaScript can access fully loaded elements, reducing dependency issues.

Q3. How can you protect against cross-site scripting (XSS) attacks when using JavaScript?

- ***To protect against Cross-Site Scripting (XSS) attacks in JavaScript:***
- ***Sanitize User Input – Use libraries like DOMPurify to clean input before rendering.***
- ***Escape Output – Encode data before inserting it into the DOM (use textContent instead of innerHTML).***
- ***Use Content Security Policy (CSP) – Restrict script execution to trusted sources.***
- ***Avoid eval() and innerHTML – These can execute malicious scripts.***
- ***Use HTTP-Only Cookies – Prevent access to sensitive cookies from JavaScript.***
- ***Validate and Sanitize on Server-Side – Ensure input is checked on the backend as well***

- ***Q4. Create an HTML table with two rows and three columns. Populate the table with data such as names, ages, and cities. Use proper HTML tags and attributes to structure the table***

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sample Table</title>
</head>
<body>

    <table border="1">
        <tr>
            <th>Name</th>
            <th>Age</th>
            <th>City</th>
        </tr>
        <tr>
            <td>Alice</td>
            <td>25</td>
            <td>New York</td>
        </tr>
        <tr>
            <td>Bob</td>
            <td>30</td>
            <td>Los Angeles</td>
        </tr>
    </table>

</body>
</html>
```

Q5. How can you parse an XML document using JavaScript? Describe with the help of suitable examples.

Steps to Parse XML in JavaScript:

1. *Create an XML string.*
2. *Use DOMParser to convert it into a DOM object.*
3. *Access and extract data using getElementsByTagName or querySelector.*

Example: Parsing an XML String

```
// Sample XML string
const xmlString = `
<employees>
  <employee>
    <name>Alice</name>
    <age>25</age>
    <city>New York</city>
  </employee>
  <employee>
    <name>Bob</name>
    <age>30</age>
    <city>Los Angeles</city>
  </employee>
</employees>`;

// Parse the XML string
const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlString, "text/xml");

// Extract data
const employees = xmlDoc.getElementsByTagName("employee");

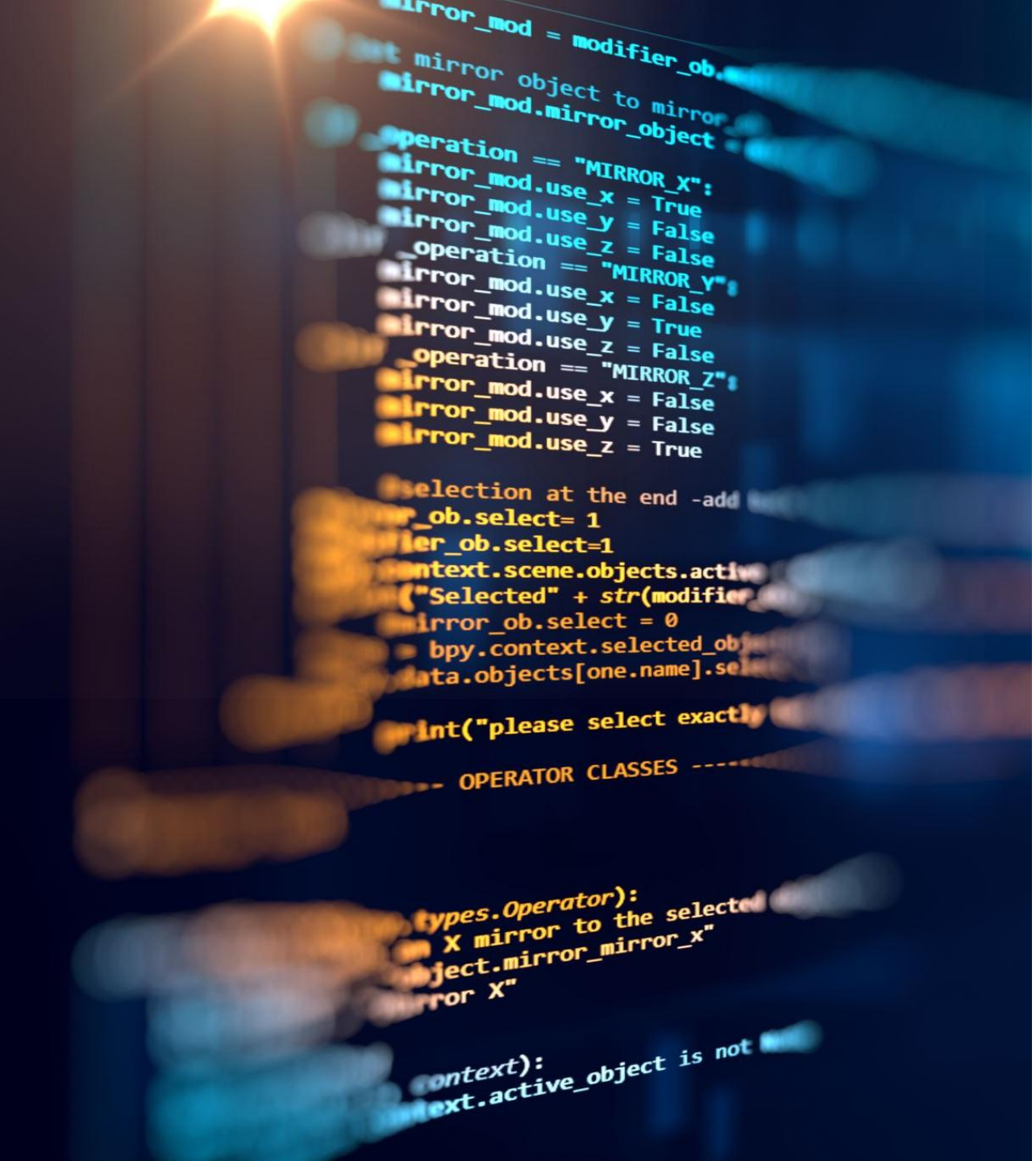
for (let i = 0; i < employees.length; i++) {
  const name = employees[i].getElementsByTagName("name")[0].textContent;
  const age = employees[i].getElementsByTagName("age")[0].textContent;
  const city = employees[i].getElementsByTagName("city")[0].textContent;
  console.log(`Employee ${i + 1}: ${name}, Age: ${age}, City: ${city}`);
```

Output:

```
: Alice, Age: 25
: Bob, Age: 30,
```

```
a.xml')
response => response.text())
xmlString => {
nst xmlDoc = new DOMParser().parseFrom
nsole.log(xmlDoc);
```

- *Alternative: Fetching XML from a File or API*
- *If fetching from an external source:*



Method 1: Using DOM API

This approach creates an XML structure dynamically:

- ***Q8. How can you generate an XML document dynamically using JavaScript?Describe with the help of suitable examples.***

```
// Create an XML document
const xmlDoc = document.implementation.createDocument("", "", null);

// Create root element
const employees = xmlDoc.createElement("employees");

// Create first employee element
const employee1 = xmlDoc.createElement("employee");

const name1 = xmlDoc.createElement("name");
name1.textContent = "Alice";

const age1 = xmlDoc.createElement("age");
age1.textContent = "25";

const city1 = xmlDoc.createElement("city");
city1.textContent = "New York";

// Append elements to employee1
employee1.appendChild(name1);
employee1.appendChild(age1);
employee1.appendChild(city1);

// Create second employee element
const employee2 = xmlDoc.createElement("employee");

const name2 = xmlDoc.createElement("name");
name2.textContent = "Bob";

const age2 = xmlDoc.createElement("age");
age2.textContent = "30";

const city2 = xmlDoc.createElement("city");
city2.textContent = "Los Angeles";
```

```
// Append elements to employee2
employee2.appendChild(name2);
employee2.appendChild(age2);
employee2.appendChild(city2);

// Append employees to root
employees.appendChild(employee1);
employees.appendChild(employee2);

// Append root to document
xmlDoc.appendChild(employees);

// Convert XML document to string
const serializer = new XMLSerializer();
const xmlString = serializer.serializeToString(xmlDoc);

console.log(xmlString);
```

```
<employees>
    <employee>
        <name>Alice</name>
        <age>25</age>
        <city>New York</city>
    </employee>
    <employee>
        <name>Bob</name>
        <age>30</age>
        <city>Los Angeles</city>
    </employee>
</employees>
```

- ***Output:***

```
const xmlString = `
<employees>
    <employee>
        <name>Alice</name>
        <age>25</age>
        <city>New York</city>
    </employee>
    <employee>
        <name>Bob</name>
        <age>30</age>
        <city>Los Angeles</city>
    </employee>
</employees>`;

console.log(xmlString);
```